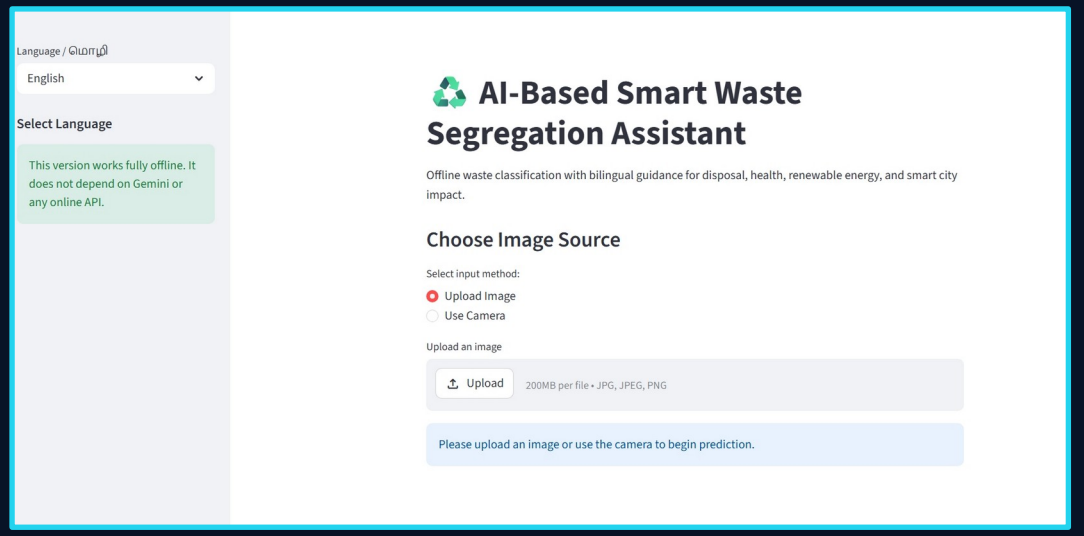
AI/ML PORTFOLIO PROJECT | COMPUTER VISION

AI-Based Smart Waste Segregation & Recycling Assistant

Image classification with transfer learning and bilingual (English / Tamil) disposal guidance

Deep Learning | Transfer Learning | Environmental AI

Thangaraj T | tmthangaraj1980@gmail.com | +91 9952045978
github.com/tmthangaraj1980-AIML



Actual app screenshot from the repository

Built at the Techies "N" Smartathon internal hackathon

Executive overview

- Upload or capture a photo of one waste item
- A CNN predicts the waste class with a confidence score
- A knowledge base shows segregation stream, disposal, recycling, health and smart-city notes
- Works offline: no online API is called
- English and Tamil interface

6

classes in the app knowledge base

224

pixel square model input

2

interface languages

1 Upload | 2 Preprocess | 3 Features | 4 Classify | 5 Guidance | 6 Display

Problem statement

Mixed waste

Wet and dry waste collected together lowers recycling quality.

Low awareness

Households are unsure which stream an item belongs to.

Pollution and health

Open dumping and burning harm air, water and public health.

Language barrier

Guidance in English alone excludes many households.

Objectives and use cases

Objectives

- Classify a waste photo
- Name the segregation stream
- Give disposal and recycling advice
- Support English and Tamil

Users

- Households
- Schools and campuses
- Municipal awareness programmes

Benefits

- Better source segregation
- Less landfill load
- Circular-economy awareness

1 Upload | 2 Preprocess | 3 Features | 4 Classify | 5 Guidance | 6 Display

Technology stack

Deep learning

- TensorFlow / Keras
- EfficientNetB0 (ImageNet weights)
- ImageDataGenerator augmentation

Application

- Streamlit web app
- Pillow, NumPy image handling
- run_app.py launcher

Knowledge base

- Python dictionary per class
- English and Tamil text
- Rule-based, not learned

Offline by design

- Model loaded from model/waste_model.keras
- No Gemini or online API call
- Camera input supported

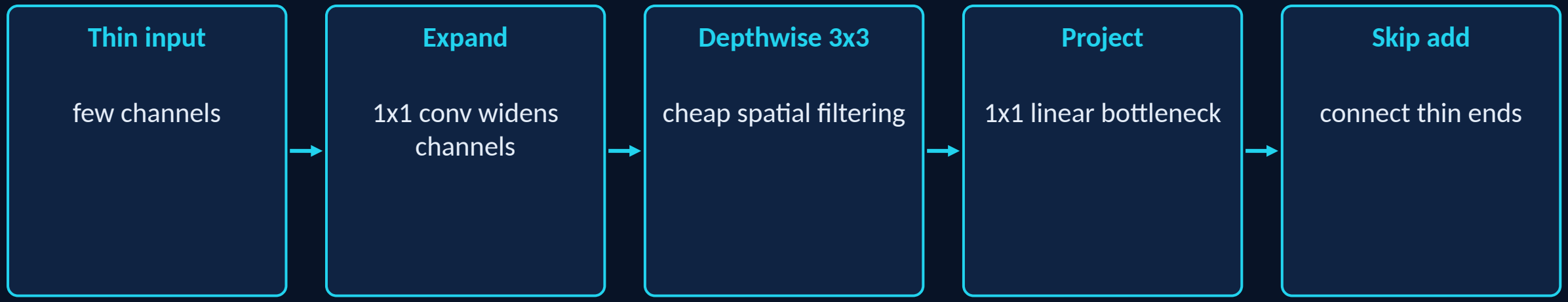
Model note: README vs source code

Item	README	Source code
Backbone	MobileNetV2	EfficientNetB0 (train_model.py)
Preprocessing	not stated	efficientnet preprocess_input
Classes	Plastic, Paper, Metal, Organic	6 keys: biological, cardboard, metal, paper, plastic, trash
Pretrained weights	not stated	ImageNet

How this deck handles it

Technical slides follow the source code (EfficientNetB0, 6 classes). MobileNetV2 is explained as the related efficient-CNN concept. Recommended fix: update the README to match the code.

Concept: efficient CNNs and inverted residuals



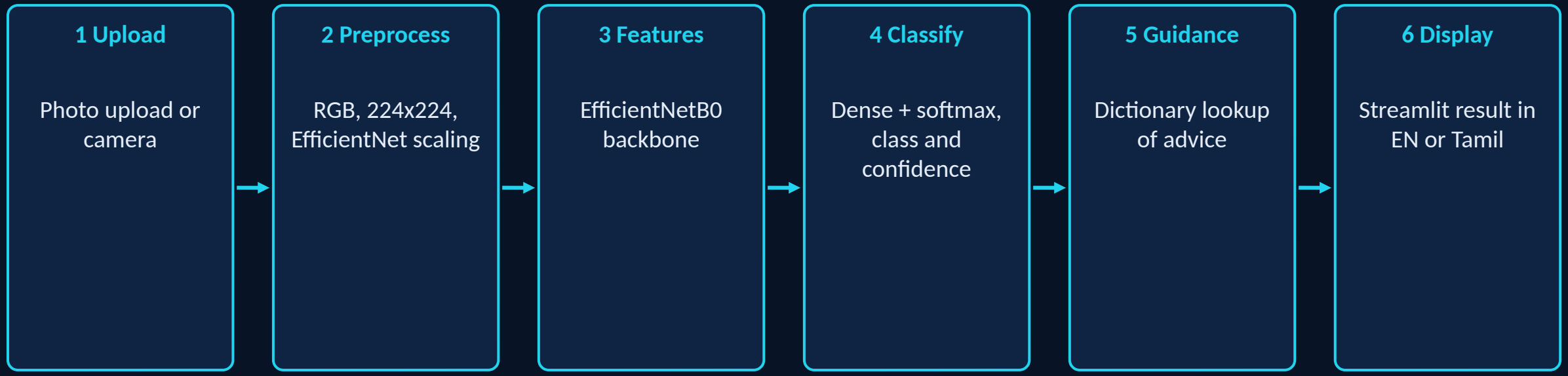
MobileNetV2

- Introduced inverted residual blocks
- Built for mobile speed

EfficientNetB0 (used here)

- Uses the MBConv block family
- Adds squeeze-and-excitation

End-to-end project flow



Stages 2 to 4 are the machine-learning part. Stage 5 is a rule-based lookup keyed by the predicted class.

System architecture



Not in the repository

Trained model file, class_names.json and the dataset folder are not committed. The repo holds code and screenshots only, so the model cannot be re-run from a clone without retraining.

1 Upload | 2 Preprocess | 3 Features | 4 Classify | 5 Guidance | 6 Display

Dataset and knowledge source

dataset/

one folder per class (not in repo)

→

flow_from_directory

labels from folder names

→

80 / 20 split

validation_split = 0.2

→

class_names.json

saved after training

Classes handled by the app

- biological, cardboard, metal
- paper, plastic, trash
- Unknown label falls back to trash

Unverified

- Dataset name and source
- Image counts per class
- Class balance

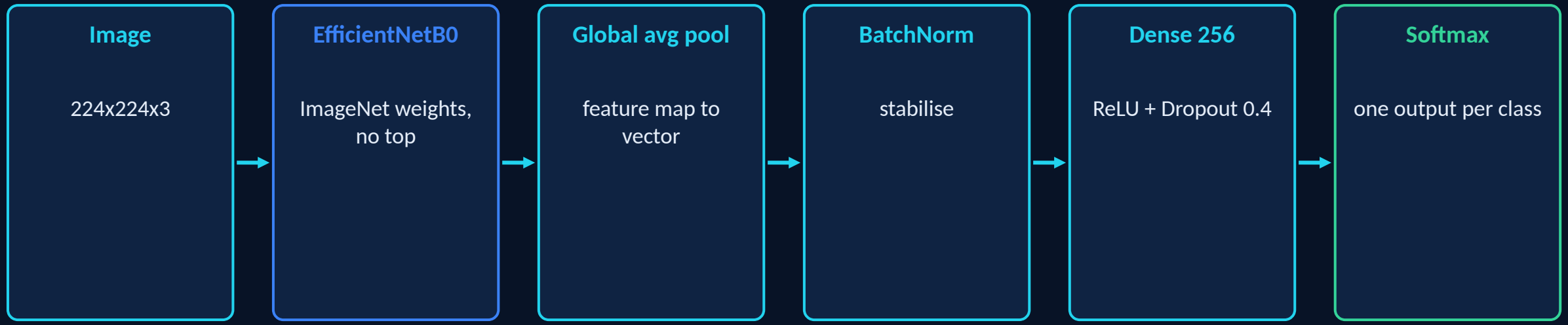
Preprocessing and augmentation

Step	Setting
Input size	224 x 224, RGB
Scaling	EfficientNet preprocess_input
Rotation / zoom	20 deg / 0.2
Shift (width, height)	0.2 / 0.2
Shear	0.15
Horizontal flip	True

Why augment?

- Few training photos rarely cover every angle
- Random changes reduce overfitting
- Applied to training data only

Model architecture



Trained here: the new head, then the last 30 backbone layers. Pretrained, not trained here: the ImageNet backbone weights.

Training strategy

Phase 1

backbone frozen, Adam 1e-3, up to 10 epochs

→

Phase 2

unfreeze last 30 layers, Adam 1e-4, 5 epochs

→

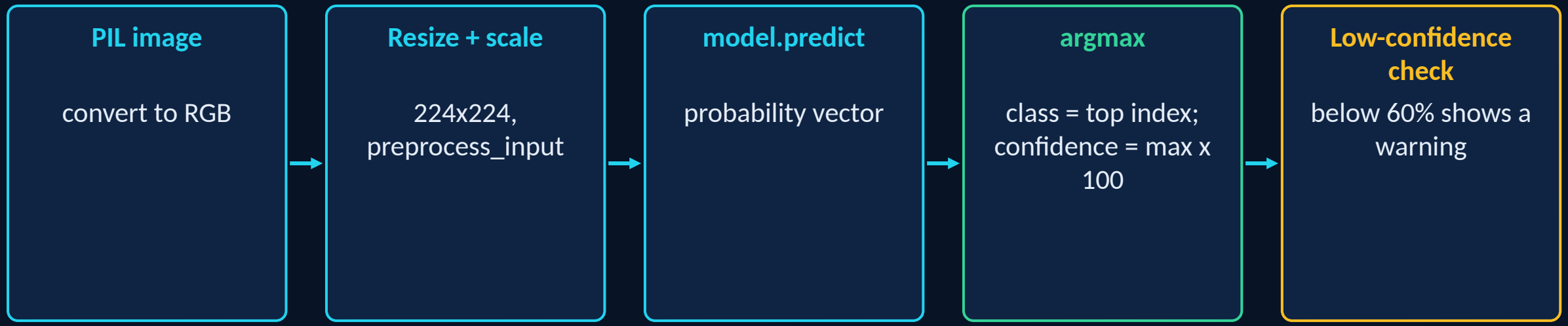
Best model

saved to model/waste_model.keras

Callbacks

- EarlyStopping on val_loss (patience 3, restore best weights)
- ReduceLROnPlateau (factor 0.2, patience 2)
- ModelCheckpoint on val_accuracy

Inference and softmax confidence



Reading confidence

97% means the model was very sure of its top class on that image, not that it is 97% accurate overall.

Stage 5: guidance is a rule-based lookup

Predicted class	Segregation stream shown
biological	Biodegradable / Wet Waste
cardboard, paper	Dry Recyclable Waste
metal, plastic	Dry Recyclable Waste
trash (and unknown labels)	Reject / Mixed Waste

Each class also has

- Disposal method
- Environmental impact
- Recycling / reuse
- Composting or energy
- Health and smart-city notes
- Verification note

Results and evaluation: what is verified

Verified

- Code trains with an 80/20 split
- App runs and returns class and confidence
- Screenshot: one prediction at 97.30% confidence (single image)

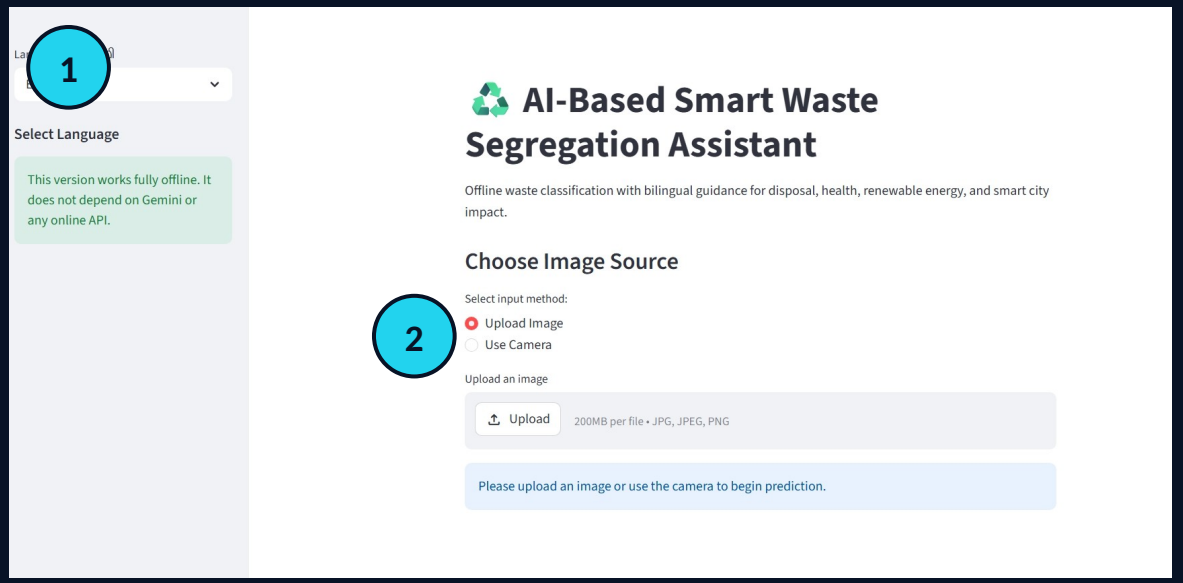
Not available

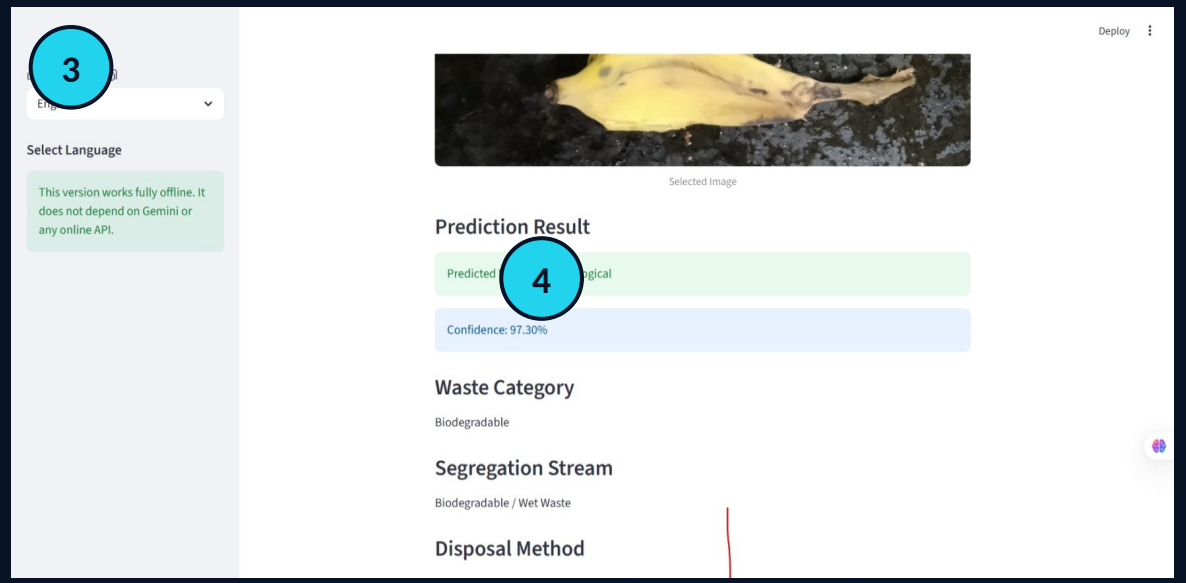
- Training / validation accuracy
- Test accuracy
- Confusion matrix
- Per-class precision and recall

Caution

- ModelCheckpoint and reporting use the same validation set
- No separate test set
- Reported numbers would be optimistic

Application demonstration



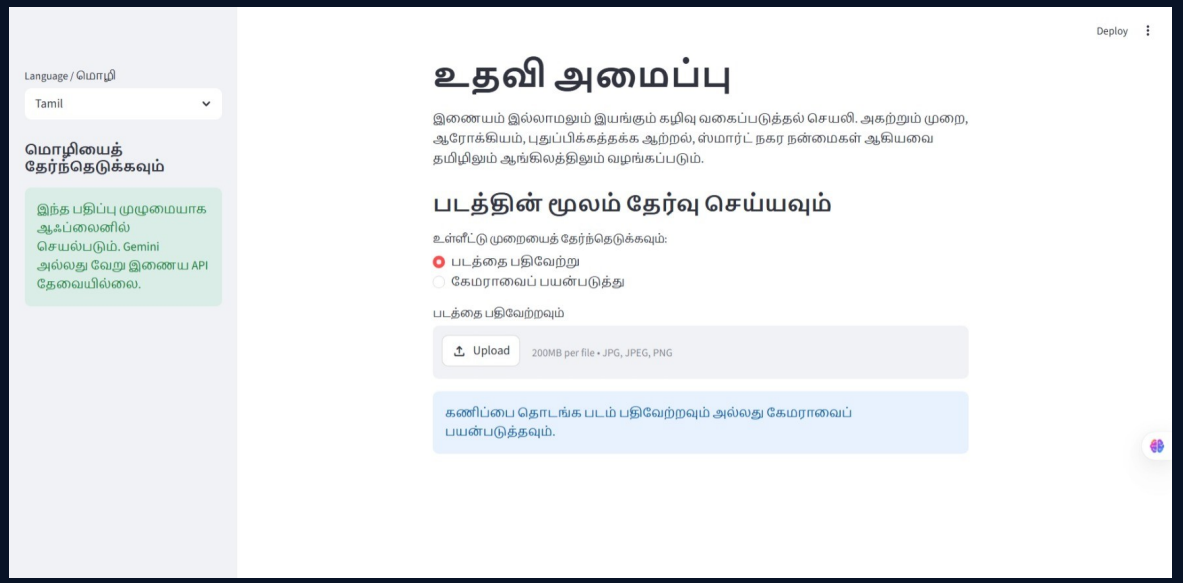


1 Language selector and offline note. **2** Upload or camera input.

3 Predicted class: Biological. **4** Confidence 97.30%, then category and stream.

Actual screenshots from the project repository

Bilingual interface (Tamil)





What it shows

Same app in Tamil: the full interface and every guidance section switch language from the sidebar dropdown. Actual screenshots from the repository.

Classification vs object detection

Image classification (this project)

- One label for the whole image
- Softmax over classes
- Best for a single item in frame
- No locations returned

Object detection (YOLO project)

- Many objects per image
- Bounding boxes + labels
- Handles mixed scenes
- Evaluated with mAP

How to evaluate this model properly

Metric	Why	How
Test accuracy	overall correctness	held-out test folder, never used in checkpointing
Confusion matrix	which classes mix up	sklearn confusion_matrix
Per-class precision / recall	class imbalance	classification_report
Real-photo test	domain shift	household photos, mixed backgrounds
Calibration	is 97% really 97%?	confidence vs accuracy bins

Technical challenges and limitations

Model

- One label per image only
- Mixed or dirty items are hard
- Confidence is not accuracy

Evidence

- No metrics committed
- Model and dataset not in repo
- README mismatch (MobileNetV2)

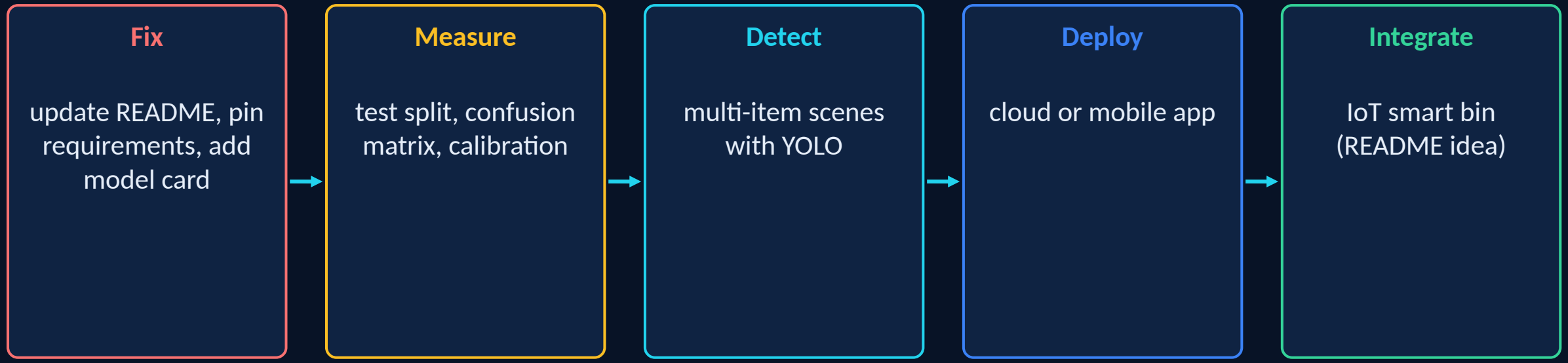
Guidance

- Rule-based text, not learned
- Local rules vary by city
- Low-confidence warning only below 60%

Packaging

- requirements.txt unpinned
- No automated tests
- Large files must be shared separately

Improvements and roadmap



Interview preparation

1. Which model does the code use?

EfficientNetB0 with ImageNet weights; the README says MobileNetV2, a mismatch I would correct.

2. Why transfer learning?

A pretrained backbone supplies visual features, so a small waste dataset trains faster with less overfitting.

3. What does the confidence mean?

The top softmax probability. It is not accuracy and can be high on a wrong class.

4. Which outputs are ML and which are rules?

Only the class and confidence are ML. Disposal and impact text is a hand-written dictionary.

5. Classification vs detection?

Classification gives one label per image; detection locates several objects with boxes, which suits mixed waste.

Resources and missing details

Available

- Repository (code, README, 4 screenshots)
- Local run: streamlit run app.py
- No live demo URL

github.com/tmthangaraj1980-AIML/AI-Based-Smart-Waste-Segregation-Recycling-Assistant

Missing or unverified

- Dataset name and size
- Trained model and class_names.json
- Accuracy and confusion matrix
- Exact class folder names

Conclusion

Skills demonstrated

- Transfer learning with Keras
- Augmentation and fine-tuning
- Streamlit app with bilingual UI
- Reading code critically against docs
- Separating ML output from rules

Project impact

- Practical aid for waste segregation
- Offline and bilingual
- Clear path to detection and IoT

Thangaraj T | tmthangaraj1980@gmail.com | +91 9952045978